

1. Введение

1.1. О чём проект

Идея пришла из задания Parsmart: сделать сервис, который по одной странице товара сам подберёт CSS-селекторы и потом будет применять их к тысячам похожих страниц. Не очередной парсер под конкретный магазин — а универсальный инструмент, работающий на Wildberries, на демо-сайтах для тестирования автоматизации и на малых WooCommerce-магазинах одинаково.

У сервиса три режима. Первый — самый простой: дать URL, получить JSON с полями товара. Здесь LLM вызывается каждый раз, поэтому это вариант для разовых проверок. Второй режим, основной по заданию, — генерация CSS-селекторов и сохранение их в формате TOON. После первой генерации все следующие страницы того же домена обрабатываются без обращения к языковой модели. Третий режим добавили в последний момент по требованию руководителя: сервис открывает страницу и сам нажимает кнопки. Например, «В корзину» на Wildberries.

1.2. Зачем это нужно

Классические парсеры — Scrapy, связка BeautifulSoup и requests — требуют, чтобы разработчик руками описал селекторы для каждого сайта. Это рутина и она ломается на любом редизайне. Когда магазин переименовал класс с product-title на ProductTitle, парсер уже не работает. Команды, занимающиеся скрейпингом, тратят на поддержку существующих парсеров примерно столько же времени, сколько на разработку новых. Большие языковые модели снимают эту проблему: они понимают, что элемент с классом product-card__price — это цена, без необходимости объяснять это разработчику.

Второй мотив проекта — современный e-commerce активно использует интерактивные элементы. Кнопки «В корзину», динамические выпадайки, инфинит-скролл с догрузкой товаров. Парсеры, читающие только статический HTML, на таких сайтах не работают. Наш модуль интерактивных действий это закрывает: можно автоматизировать тестирование интернет-магазина, имитировать действия пользователя для маркетинговых исследований или встроить парсер в QA-пайплайн.

1.3. Цель

Собрать сервис, который умеет: подбирать CSS-селекторы по одной странице через LLM, применять их без LLM ко всем остальным страницам того же домена, и взаимодействовать со страницей (клики по кнопкам). Всё это — без жёстких правил под конкретный магазин и с открытым кодом.

1.4. Задачи

Что было нужно сделать:

Изучить, что уже есть на рынке — Scrapy, Octoparse, ParseHub, Diffbot, ScrapingBee, Arify. Найти нишу, которую никто не закрывает.

Спроектировать архитектуру на FastAPI: асинхронный пайплайн загрузки HTML, очистки, обращения к LLM и сохранения результата.

Написать модуль генерации CSS-селекторов и сериализатор в формат TOON — он же из задания Parsmart.

Сделать модуль интерактивных действий с универсальным поиском кнопок (требование руководителя на финале).

Подключить SQLite через aiosqlite для истории запросов и сохранения разметок.

Покрыть всё тестами с моками Playwright и LLM, чтобы тесты прогонялись без сети.

Прогнать сервис на синтетических фикстурах и на реальных сайтах, включая Wildberries, Oзон, DNS-shop.

Выложить проект на GitHub, написать README и техническую документацию.

2. Обзор аналогов

На первом этапе работы Анна разобрала рынок инструментов для веб-парсинга. Получилось три группы: классические парсеры с жёсткими правилами, no-code платформы с визуальным конструктором и AI-сервисы на машинном обучении.

2.1. Классические парсеры

Scrapy

Главный open-source фреймворк для скрейпинга в Python. Сильное сообщество, тысячи плагинов, асинхронная архитектура поверх Twisted. На бумаге всё хорошо.

Но Scrapy требует, чтобы разработчик руками описал XPath или CSS-селекторы для каждого магазина. При любом редизайне сайта парсер ломается. SPA на React или Vue без отдельной интеграции со Splash либо Playwright не открывает. То есть Scrapy — это инструмент для специалиста, а нам нужна универсальность. Наш сервис подбирает селекторы сам, через LLM.

BeautifulSoup и requests

Не фреймворк, а низкоуровневая библиотека для парсинга HTML. Простая, гибкая, любимая всеми. Но работает только на статичных страницах и весь код извлечения пишет разработчик. В нашем проекте BeautifulSoup используется внутри — но как инструмент применения уже сгенерированных LLM-селекторов, а не как ядро.

2.2. No-code платформы

Octoparse

Десктоп-приложение с встроенным браузером. Пользователь кликает по элементам, программа сама предлагает селекторы, всё работает визуально. Из плюсов — облачное выполнение, есть API.

Минусы: закрытый код, платные тарифы, при редизайне сайта пользователь снова вручную перестраивает шаги. Никакой автоматической миграции на новый макет. LLM Web Parsing в этом месте побеждает: чтобы обновить разметку, достаточно вызвать генерацию заново — кликать ни по чему не нужно.

ParseHub

Конкурент Octoparse, плюс-минус то же самое. Дополнительная проблема — нет нормального self-hosted режима, всё выполняется только на их серверах.

Apify

Облачная платформа с маркетплейсом «акторов» — готовых парсеров под популярные сайты. Десятки тысяч решений из коробки, прокси встроены. Но это всё облачное, платное и закрытое. У нас self-hosted, без привязки к вендору.

2.3. AI-решения

Diffbot

Коммерческий сервис с собственным компьютерным зрением и ML-моделями. Качество извлечения очень высокое, поддерживают не только товары, но и статьи, профили, видео. Минусы — от 299 долларов в месяц и они отдадут только JSON с данными, никаких CSS-селекторов. То есть пользователь не может встроить их результат в свой Scraper-парсер. У нас наоборот — мы отдаём именно селекторы, и их можно скопировать куда угодно. И бесплатно.

ScrapingBee

Browser-as-a-service: дают облачный браузер и можно отправить ему промпт для AI-извлечения. Сами селекторы не сохраняются — каждый запрос это новый вызов LLM, значит дорого. У нас один вызов модели на домен, дальше всё крутится на BeautifulSoup.

2.4. Что в итоге

Ни одно из решений не сочетает четыре вещи: LLM для разметки, выдачу селекторов наружу (а не только данных), open-source с self-hosted деплоем и интерактивные действия на странице. Эту нишу мы и закрыли. По стоимости — бесплатно, по скорости — на уровне классических парсеров, но гибкости — на уровне AI-сервисов.

3. Архитектура

3.1. Как устроен пайплайн

Берём URL. Playwright открывает страницу в headless Chromium, запускаем stealth-patch и отдаёт нам HTML. Дальше BeautifulSoup выкидывает script, style, footer и прочий мусор — размер документа падает в 10–20 раз. Очищенный HTML уходит в LangChain, тот вызывает Llama 3.1 или GPT-4o-mini. Ответ модели проходит через Pydantic-валидацию, и финальные данные ложатся в SQLite через aiosqlite.

В режиме CSS-селекторов LLM спрашиваем один раз — для опорной страницы домена. Модель возвращает JSON с шестью селекторами: name, price, currency, pictures, category, article. Дальше каждый селектор проверяем на исходной странице: если BeautifulSoup ничего по нему не находит, сбрасываем в null и пишем предупреждение в markdownComments. Это защита от галлюцинаций — модель иногда придумывает классы, которых на странице нет вообще. После проверки разметка сохраняется в таблицу domain_markup и используется для всех следующих URL того же магазина без обращения к LLM.

В режиме интерактивных действий пайплайн другой. Playwright открывает страницу. Дальше element_finder проходится по DOM, оценивает кандидатов на роль нужной кнопки по тексту, aria-label, классу, role и тегу. Если уверенность скоринга получается ниже 30 баллов, спрашиваем LLM — пусть подскажет селектор. Если LLM тоже не уверен (confidence < 50), честно возвращаем not_found. Если уверен — кликаем через Playwright и ждём, как страница реагирует.

3.2. Стек

Python 3.12. Бекенд — FastAPI 0.115 с Uvicorn (он же ASGI-сервер). Валидация — Pydantic 2.9. База — SQLite через aiosqlite 0.20, три таблицы: parsed_products, domain_markup, interaction_runs. LLM — LangChain 0.3 поверх huggingface-hub 1.16, с fallback на прямой вызов InferenceClient. Опционально langchain-openai для GPT-4o-mini. Браузер — Playwright 1.47, headless Chromium, с playwright-stealth 2.0 поверх. Очистка HTML — BeautifulSoup4 4.12 + lxml

5.3. Тесты — `pytest 8.3` с `pytest-asyncio` и `httpx` для интеграции. Денплой — `Docker` и `docker-compose`, поднимается одной командой.

3.3. Что мы делали правильно

Разделили код по слоям. У каждого модуля одна задача: `fetcher` грузит HTML, `cleaner` чистит, `llm.parser` ходит в модель, `applier` применяет селекторы, `models.schemas` хранит Pydantic-классы, `database.db` работает с SQLite, `toon.serializer` рисует TOON-формат, `api.routes` склеивает всё в HTTP-эндпоинты. Это банальная история, но она реально упрощает тесты — каждый модуль покрывается отдельно.

Сделали ленивую инициализацию LLM-клиента. Соединение с Hugging Face устанавливается не на старте приложения, а при первом запросе. Это позволяет в тестах создавать LLMParser с ключом-заглушкой и мокать `ainvoke`. Никаких реальных обращений в сеть во время прогона тестов.

Сделали fallback на прямой Hugging Face API. LangChain любит ломать обратную совместимость между минорными версиями — это история на отдельную статью. Когда стандартная цепочка `template | llm` падает с непонятной ошибкой, мы вызываем `huggingface_hub.InferenceClient.chat_completion` напрямую. Сервис продолжает работать.

Написали толерантный JSON-парсер. Модели иногда обрамляют ответ в markdown-блок, ставят одинарные кавычки, добавляют trailing comma, пишут True/False вместо true/false. Парсер сначала пробует `json.loads` напрямую, не получилось — снимает markdown, заменяет кавычки, чистит запятые. Грязно, но работает.

Главное архитектурное решение: кэшируем не сырые данные, а именно селекторы. Один вызов модели на домен покрывает тысячи страниц. Если на странице товара поменялась цена — наш селектор найдёт новое значение без новой генерации. А если магазин сделал редизайн, достаточно дёрнуть `POST /api/selectors/generate` ещё раз.

4. Вклад участников

4.1. Широкая Анна

4.1.1. Что я делала

Моя ответственность — план работ, координация и аналитика. На старте я разобрала рынок инструментов веб-парсинга, сравнила сильные и слабые стороны существующих решений, и из анализа уже выросла идея проекта. На финале я ещё раз прошла по конкурентам — Diffbot, ScrapingBee, Apify, Octoparse — и написала сравнение в раздел отчёта.

Главный вывод из обзора был такой. Все классические парсеры ломаются на редизайнах. Все no-code платформы требуют, чтобы пользователь кликал руками. AI-сервисы дорогие и закрытые. Никто не отдаёт сами селекторы наружу — а это нужно, если хочешь использовать результат в своих Scraper-наплайнах. Так что ниша есть.

Я предложила архитектуру: REST API на FastAPI как точка входа, Playwright для загрузки страниц со stealth-патчами, BeautifulSoup для очистки HTML, LangChain для общения с моделью, Pydantic для валидации, aiosqlite для хранения. Каждый компонент — отдельный модуль с одной задачей.

Технологический стек тоже на мне. Python 3.12 — потому что у него зрелая экосистема для машинного обучения и веба. Hugging Face Inference API — бесплатно, доступ к Llama 3.1 8B без оплаты. OpenAI GPT-4o-mini — опционально, для случаев когда нужно качество. Выбор между провайдерами происходит автоматически по наличию ключа.

По командной работе — мы синхронились раз в неделю по часу. Распределяли задачи, я следила чтобы критичные модули не застряли. На финальном этапе, когда руководитель попросил дописать интерактивные действия, я перепроектировала схему взаимодействия модулей: `element_finder` Даниила должен был встать рядом с `interactor` Михаила, а `cleaner` — научиться работать в двух режимах. Это потребовало пары встреч.

4.1.2. С чем были проблемы

Главный вопрос на старте — какую модель брать. OpenAI давал бы лучшее качество, но за деньги. Hugging Face бесплатный, но ограничения по rate-limit и медленнее. Решили так: пишем архитектуру с двумя провайдерами сразу, выбор по переменной `OPENAI_API_KEY`. Есть ключ — берём OpenAI, нет — Hugging Face.

Второй сложный момент — размер HTML. У Wildberries страница товара легко тянет на 1.6 МБ. В контекст бесплатных моделей столько не влезает. Договорились с Михаилом, что `cleaner` будет максимально агрессивным: всё лишнее выкидывает, для самых длинных страниц — адаптивное усеечение (берём начало и конец, середину отрезаем).

Третий вопрос — отказоустойчивость. Не все запросы доходят с первого раза. Сеть может моргнуть, Playwright может упасть, LLM может вернуть мусор. Договорились про `tenacity` для `retry` с экспоненциальной задержкой, про таймауты на все этапы и про то, что

любая ошибка должна возвращать `status=error` в JSON, а не 500-й код HTTP. Иначе клиенту неудобно работать.

4.1.3. Что дальше

Хочется собрать тестовый датасет на 200–300 страниц с разметкой от руки, чтобы можно было считать точность по разным категориям товаров. Сейчас мы тестируем на десятке сайтов, и это маловато. Ещё нужна система мониторинга качества в проде — чтобы автоматически замечать, когда селекторы перестали работать (магазин сделал редизайн), и триггерить генерацию заново.

4.2. Копейкин Михаил

4.2.1. Что я делал

Я отвечал за бекенд и за всю инфраструктуру парсинга. Это значит — FastAPI и эндпоинты, загрузчик страниц на Playwright, препроцессор HTML, интеграция с LLM через LangChain, и модуль `interactor` для интерактивных действий, который мы написали на финальной стадии.

Начал с REST API. FastAPI вместо Flask — потому что нативный `async/await`, автоматический Swagger UI на `/docs`, валидация через Pydantic из коробки. Получились эндпоинты `POST /api/parse` для прямого извлечения, `GET /api/results` для истории, `POST /api/selectors/generate` для генерации селекторов (но саму логику селекторов писал Даниил).

Дальше `fetcher`. Playwright в `headless`-режиме, Chromium 121. Простой `requests` не подошёл бы — у Wildberries карточка товара рисуется JavaScript-ом из API, без рендера React ничего не покажет. В `fetcher.py` есть `retry` на трёх попытках через `tenacity`, с экспоненциальной паузой от 2 до 10 секунд. Если `networkidle` не пришёл за минуту — продолжаем (на крупных SPA он не приходит вообще, `polling` никогда не затухает).

Cleaner писал я. По умолчанию режу всё подряд: `script`, `style`, `noscript`, `iframe`, `svg`, `meta`, `link`, `head`, `footer`, `nav`, `aside`, `header`, `form`, `button`, `input`. Остаётся только семантическое ядро страницы. Атрибуты тоже зачищаю — оставляю `class`, `src`, `href`, `alt`, `id`, `data-article`, `data-sku`. Дальше итеративно выкидываю пустые контейнеры до стабилизации. Когда добавляли интерактивные действия, пришлось переделывать: добавил флаг `keep_interactive=True`, при котором `button`, `input` и `form` остаются, и плюс атрибуты `aria-label`, `role`, `data-action`, `onclick`, `disabled` — без них `element_finder` ничего не находит.

LLM-парсер построен поверх LangChain. Создаю `PromptTemplate` (имя, описание, набор переменных), компонирую с моделью оператором `pipe`: `chain = template | llm`. Дальше `ainvoke` и получаем ответ. Температура нулевая, чтобы было детерминированно (вернее почти нулевая — Hugging Face Endpoint не любит чистый ноль, поставил 0.01). Максимум 1024 токена на ответ.

Самое мерзкое в LLM-слое — это нестабильность LangChain. Между версиями ломают сигнатуры, `ImportError` выпадает на пустом месте. Поэтому встроил двухуровневый `fallback`. На первом уровне стандартная цепочка `template | llm`. Если падает — на втором уровне сразу из своего кода зову `huggingface_hub.InferenceClient.chat_completion`. Это банальный POST на HF, без всяких абстракций. Работает всегда, когда сам HF доступен.

На финальном этапе делал `interactor` — модуль интерактивных действий. По заданию руководителя сервиса должен не только читать страницу, но и нажимать кнопки, добавлять в корзину. Алгоритм поиска самих кнопок (эвристика с системой скоринга) разрабатывал Даниил — я взял его `element_finder` и обернул в Playwright-логику. То есть моя часть — открыть страницу, дождаться `domcontentloaded`, потом по возможности `networkidle`, опционально подождать конкретный селектор. Потом отдать очищенный HTML в `element_finder`, получить от него селектор и кликнуть. После клика подождать обновления страницы, считать новый `title`, всё сохранить в БД с детальным логом по шагам (`timestamp_ms` и `step name`).

4.2.2. Где сидел в багах

Самая длинная история — антибот-защита крупных маркетплейсов. Wildberries по умолчанию возвращает почти пустой `shell`, потому что весь контент догружается JS через несколько секунд после `networkidle`. Сначала я думал что наши UA палятся, но даже с корректным User-Agent ничего не загружалось. В итоге интегрировал `playwright-stealth` — пакет, который патчит `navigator.webdriver`, эмулирует Chrome runtime, подменяет WebGL fingerprint. Плюс добавил полный набор Chrome-заголовков (`Sec-Ch-Ua`, `Sec-Fetch-*`, `Accept-Language=ru-RU`) и `timezone Europe/Moscow`. После этого WB начал отдавать нормальный HTML.

DNS-shop и Ozon не прошли. У них либо DataDome с TLS-fingerprint, либо капча сразу. Это известная история — даже коммерческие сервисы вроде Diffbot решают это резидентными прокси. Self-hosted без прокси-сети тут не вывезет, и мы это написали в раздел про ограничения.

Вторая беда — JSON от языковых моделей. Иногда ответ обернут в тройные обратные кавычки с пометкой json, иногда LLM ставит trailing comma, иногда пишет 'title' одинарными кавычками вместо двойных, иногда вместо null идёт None как в Python. Сделал функцию `_extract_json` которая последовательно лечит все эти кейсы: снимает markdown, чистит запятые, заменяет кавычки. Это вообще одна из самых тестируемых вещей в проекте, `test_llm_parser.py` покрывает 12 разных вариантов мусора.

Третья проблема — `networkidle` на современных сайтах никогда не наступает. Бесконечный polling аналитики, WebSocket к чату поддержки и так далее. Сначала ловил `TimeoutError` и кидал исключение, потом понял что это нормальная ситуация. Сделал двухступенчатое ожидание: гарантированно ждём `domcontentloaded`, а потом пытаемся `networkidle` с тайм-аутом 10 секунд, но не падаем если он не пришёл.

4.2.3. Куда расти

Хочу попробовать параллельную обработку: несколько URL одного домена в одном `browser-context` с переиспользованием cookie. Сейчас каждый запрос поднимает свой Chromium, это медленно. Для прода нужен PostgreSQL вместо SQLite — переход тривиальный, интерфейс модуля `database.db` специально абстрактный. И ещё интерактивные действия можно расширить: сейчас один клик за раз, а хочется multistep — например, выбрать размер, потом цвет, потом добавить в корзину.

4.3. Беляев Даниил

4.3.1. Что я делал

На мне три большие зоны кода. Первая — модуль CSS-селекторов и сериализатор TOON, это центральная фишка по заданию Parsmart. Вторая — слой данных: Pydantic-модели для валидации и асинхронный доступ к SQLite. Третья — эвристика поиска интерактивных элементов, которую Михаил позже интегрировал в `interactor`. Плюс на мне всё, что связано с тестированием — методология, автотесты для моей зоны, прогоны на реальных сайтах.

4.3.2. CSS-селекторы и TOON

Это сердце проекта. Пользователь даёт нам URL одной товарной карточки, мы возвращаем CSS-селекторы для имени, цены, артикула, фото, категории, валюты. Дальше эти селекторы применяются ко всем остальным карточкам того же магазина уже без LLM. По коду это три файла: `selectors.py` (генерация и валидация), `applier.py` (применение через BeautifulSoup без LLM), `toon/serializer.py` (сериализация в формат TOON из T3).

Промпт для генератора писал я. Получилось не сразу — первые версии возвращали почти случайные классы. Помогли явные инструкции внутри промпта: «бери класс- и атрибут-селекторы», «избегай авто-генерированных хэшей вроде `css-1a2b3c`», «используй только реально присутствующие в HTML классы». Стало стабильнее, но иногда LLM всё равно фантазирует.

Чтобы это поймать, написал валидатор. Функция `validate_selectors` берёт каждый сгенерированный селектор и пробует найти по нему элемент в исходном HTML через BeautifulSoup. Если найден — ок, оставляем. Если ничего не нашёл или CSS невалидный — сбрасываем в `null` и записываем в поле `markdownComments` человекочитаемую причину. Пользователь потом видит в TOON-файле: `<price: селектор 'css-1a2b3c' ничего не нашёл>`. Это превращает галлюцинации модели из бага в фишку — есть явная обратная связь.

Модуль `applier` применяет сохранённые селекторы к новым URL без обращения к LLM. Это просто BeautifulSoup, но с нормализацией данных. Функция `_normalize_price` парсит разные форматы: `<1 999 Р>` → 1999.0, `<1 999,99 Р>` → 1999.99, европейский `<1.999,99 Р>` → 1999.99. Функция `_normalize_article` склеивает пробелы внутри числовых артикулов (`<2006 4875>` → `<20064875>`), но сохраняет alphanumeric SKU как есть (`<SM-A55>`, `<LEN-001>`). Функция `_absolutize` превращает относительные URL картинок в абсолютные.

TOON-сериализатор я писал с нуля. Формат не YAML и не JSON — массивы там объявляются с указанием длины: `markdownComments[3]` вместо просто `markdownComments`, и элементы нумеруются `<1: text>`, `<2: text>`, `<3: text>`. Никаких готовых библиотек для TOON нет, поэтому функция `dump_domains` и её помощники `_write_domain`, `_write_product_page` и так далее написаны вручную. Особое внимание — порядок полей в `productPage`: `category`, `currenasy`, `name`, `pictures`, `price`, `article`. Точно такой же порядок в эталонном `domains.toon` из задания, я этим долго сверялся.

4.3.3. Pydantic и SQLite

Все межслойные контракты — Pydantic-модели в `app/models/schemas.py`. На вход API принимает `ParseRequest` (URL и source), `ProductData` описывает результат, `DomainMarkup` — полную разметку домена для TOON, и так далее. Когда добавляли модуль интерактивных действий, появились ещё `InteractRequest` и `InteractResponse`.

Кастомные валидаторы я подбирал на боевых ошибках. Например, валидатор цены: минимум 0, максимум 10 миллионов, округление до двух знаков. Раньше пропускал любое число,

но LLM иногда возвращал отрицательные значения (видимо, путал с скидкой). Теперь падает с `ValidationError`, а вызывающий код перехватывает это и пишет в БД `status=error`. Валидатор URL изображения требует схему `http(s)://` и возвращает `None` для мусора. Валидатор title режет всё длиннее 500 символов и превращает пустые строки в `None`.

Хранилище — `SQLite` через `aiosqlite`. `PostgreSQL` для курсового избыточен, а `aiosqlite` даёт нативный `async`, что важно для `FastAPI`. Три таблицы: `parsed_products` (история прямых извлечений), `domain_markup` (сохранённые CSS-разметки), `interaction_runs` (журнал интерактивных действий). На `parsed_products` и `interaction_runs` повесил индексы по `url` и `created_at` — нужны для нормального постраничного вывода истории.

Особый момент в `db.py` — функция `save_domain_markup`, она делает `UPSERT` через `ON CONFLICT`. То есть если разметка для домена уже есть и пользователь дёргает `/selectors/generate` ещё раз (например, сайт переехал), старая запись перезаписывается атомарно. Без `UPSERT` пришлось бы делать `SELECT`, проверять, потом `INSERT` или `UPDATE` — это две лишние round-trip и потенциальная гонка.

4.3.4. Эвристика поиска кнопок

На финальной стадии, когда руководитель попросил интерактивные действия, моя задача была — придумать, как универсально искать кнопки на любом интернет-магазине без жёстких правил под конкретный сайт. Получился модуль `element_finder.py` — система эвристического скоринга.

Идея простая: проходимся по всем потенциально-кликабельным элементам на странице (`button`, `a`, `input[type=submit]`, `div` с `role=button` или `onclick`), и для каждого считаем балл по семи признакам. Точное совпадение текста с фразой намерения — 20 баллов. Текст содержит фразу — 15. Все слова фразы встречаются — 8. `aria-label` оценивается с коэффициентом 0.9 от текста, `title` — с коэффициентом 0.7. Класс или `id`, соответствующий регулярному выражению из словаря (например, `/add[-_]?to[-_]?cart/`) — 25 баллов. `role=button` — 5. Тег `button` сам по себе — 10, `input[type=submit]` — 8, `a` — 4.

Отдельный кусок логики — `anti-patterns`. «В избранное», «wishlist», «compare», «notify me» снижают балл на 50. Без этого скоринг ошибался: кнопка «Добавить в избранное» по тексту тоже содержит «Добавить», и LLM её путал с корзиной. `Disabled`-кнопки минус 30. Это нужно, чтобы серая «Закончился» не побеждала активную «В корзину».

Многоязычность — словарь `ADD_TO_CART_TEXTS`. Русский: «в корзину», «купить», «заказать», «положить в корзину». Английский: «add to cart», «add to bag», «buy now», «purchase». Французский: «ajouter au panier». Немецкий: «in den warenkorb». Добавить новый язык — одна строка в словаре.

Минимальный порог — 30 баллов. Меньше — считаем что эвристика не уверена. В этом случае Михаил в `interactor` дёргает `LLM-fallback`: тот же HTML отправляется в модель с просьбой вернуть селектор и `confidence`. Если `confidence` < 50, возвращаем `not_found`. Это важно — лучше честно сказать «не нашёл», чем кликнуть наугад.

Отдельная функция `build_selector` собирает уникальный CSS-селектор для найденного элемента. Приоритеты: сначала `id` (если не похож на авто-генерированный хэш), потом `data-testid`, `data-action`, `input[name=...]`, потом `tag` с осмысленными классами, в крайнем случае — путь от ближайшего предка с осмысленным `id`. Главное — отсекай хэш-классы `CSS-in-JS`, `styled-components`, `JSX`, `Material-UI`. Функция `looks_autogenerated` по регуляркам ловит `css-1a2b3c`, `sc-aBcD1`, `jsx-1234567890`, `MuiButton-root-12345` — все эти штуки меняются при каждой сборке сайта и приводят к нестабильным селекторам.

4.3.5. Тестирование

Подход трёхуровневый. `Unit` — отдельные функции без сети и БД. `Integration` — реальный `FastAPI` и временная `SQLite`, но `Playwright` и `LLM API` замочаны через `unittest.mock`. `End-to-end` — ручные прогоны через работающий сервер на синтетике и реальных сайтах.

Всего получилось 139 автотестов в 10 модулях. Прогоняются за секунду — потому что внешние API замочаны, никаких сетевых запросов. Это позволяет крутить тесты в CI на каждом коммите. Моими руками написаны тесты, относящиеся к моим модулям: `test_schemas.py` (16 тестов Pydantic-валидации), `test_applier.py` (19 тестов применения CSS и нормализации цены/артикула), `test_toon.py` (12 тестов сериализатора), `test_database.py` (8 тестов `SQLite` CRUD), `test_selectors_api.py` (9 тестов `/selectors/*` эндпоинтов), `test_element_finder.py` (20 тестов скоринга). Тесты для модулей Михаила писали вместе — он набрасывал каркас, я делал `code review` и дорабатывал.

Из `end-to-end` самое интересное — `books.toscrape.com`. Это официальный демо-сайт для тестирования парсеров. Сгенерировал селекторы по странице «A Light in the Attic», получил корректные `.product_page h1` и `.product_main .price_color`. Применил эти же селекторы к ДРУГОЙ книге того же домена — «Tipping the Velvet». Без вызова LLM, чистый `BeautifulSoup`.

Получил `title=«Tipping the Velvet», price=53.74, article=«90fa61229261140a»`. Это в точности то поведение, которое описано в задании Parsmart.

На Wildberries протестировали интерактивное действие. URL `/catalog/192222089/detail.aspx, action=add_to_cart`. Эвристика дала score 29 — это ниже порога 30, поэтому пошёл LLM-fallback. GPT-4o-mini вернул селектор `.product-card_add-basket.j-add-to-basket` с confidence 90 и обоснованием «This element has a clear class name indicating it is the add-to-basket button». Клик прошёл за 16 секунд.

4.3.6. Где спотыкался

Дольше всего возился с галлюцинациями LLM при генерации селекторов. На страницах с сотнями классов в DOM модель иногда придумывала классы, которых там нет. Сначала просто ловил пустые результаты в апплаере и не понимал, в чём дело. Потом сделал валидатор — теперь сразу видно, что селектор выдуман, и он отбрасывается.

С Pydantic был неочевидный момент с custom-валидаторами. Хотелось одновременно и нормализовать значение (например, распарсить «1 999,99 ₽» в float), и провалидировать. Сначала разделял на два валидатора — один pre-обработка, второй проверка диапазона. Потом понял что это лишнее: можно один field_validator с режимом `mode='before'` который сразу делает обе вещи. Стало чище.

При проектировании эвристики поиска кнопок самым сложным оказался подбор весов. Что важнее: точное совпадение текста (20 баллов) или класс-паттерн (25)? Поэкспериментировал на десятке реальных сайтов: пришёл к тому, что текст всё-таки должен побеждать класс на коротких страницах, а класс — на сложных. В итоге оба сигнала складываются (не максимум), плюс anti-patterns с весом -50 — этого достаточно, чтобы «В избранное» никогда не побеждала «В корзину».

На реальных маркетплейсах ground truth субъективен. На странице iPhone в Wildberries есть и основной товар, и блок «Все 68 предложений от 70 120 ₽» с альтернативными продавцами. Что считать правильным title? Я выбрал — главный из h1, остальные игнорируем. Это записал в test-фикстуры и критерии разметки. Без чёткого критерия результаты считать невозможно — для одной и той же страницы получается «70% точности» или «20%» в зависимости от того, как разметить.

4.3.7. Что хочется доделать

По CSS-селекторам — научить генератор работать с cartPage и orderPage, не только с productPage. Сейчас модели ProductPageSelectors есть, а CartPageSelectors и OrderPageSelectors объявлены, но генератор для них не написан. Нужен ещё один промпт и логика автоматической классификации страницы — карточка товара или корзина.

По слою данных — миграция на PostgreSQL для прод-нагрузок. Интерфейс модуля db.py написан абстрактно, замена aioredis на asyncpg займёт пару часов. Плюс кэш в Redis для горячих разметок.

По эвристике — добавить новые intent'ы. Сейчас три: add_to_cart, buy_now, open_product. Хочется ещё leave_review, select_option (выбор размера/цвета), apply_filter в каталоге. Каждый — пять строк в словаре INTENTS, в основном коде ничего менять не надо.

По тестированию — собрать benchmark на разных моделях. Сравнить Llama 3.1 8B, Llama 3.1 70B, Mistral 7B, GPT-4o-mini, GPT-4o, Claude 3.5 Haiku на одном датасете. Прикинуть стоимость и точность каждой. Также эксперимент с few-shot learning — добавить в промпт примеры удачных селекторов и посмотреть, насколько лучшие станут результаты.

5. Реализация ключевых компонентов

5.1. REST API

FastAPI выбрали из-за нативного async, автоматического Swagger UI и интеграции с Pydantic. Эндпоинты разделены на четыре группы. Прямое извлечение: `POST /api/parse, GET /api/results, GET /api/results/{id}, DELETE /api/results`. CSS-селекторы по заданию Parsmart: `POST /api/selectors/generate, POST /api/selectors/apply, GET /api/selectors, GET /api/selectors/export.toon` (выгрузка в TOON-формате как в задании). Интерактивные действия: `POST /api/interact, POST /api/cart/add` (шорткат для самого популярного действия), `GET /api/interactions/runs, GET /api/interactions/runs/{id}`. Служебные: `GET /health, POST /api/interactions`.

5.2. Очистка HTML

У типичной страницы Wildberries сырой HTML — 1.6 МБ. В контекст бесплатной Llama 3.1 8B такое не помещается. BeautifulSoup отжимает документ до 12 тысяч символов в обычном режиме и до 20 тысяч в интерактивном.

Алгоритм такой: сначала вырезаем мези-помойку (script, style, noscript, iframe, svg, meta, link, head, footer, nav, aside, header). В обычном режиме сюда же идём form, button, input — они мешают извлечению. В интерактивном режиме наоборот: form/button/input сохраняем, плюс атрибуты aria-label, role, data-action, onclick, disabled — без них element_finder не работает. Дальше у оставшихся тегов чистим атрибуты — оставляем только class, src, href, alt, id и data-

*. Итеративно удаляем пустые контейнеры. В конце сжимаем пробелы и обрезаем до лимита (для длинных страниц берём начало и конец, середину выкидываем — там обычно реклама и блок «похожие товары»).

5.3. CSS-селекторы и TOON

Главное архитектурное решение проекта — кэшировать CSS-селекторы, а не сами данные. LLM спрашиваем один раз на домен. На вход опорная страница товара, на выходе JSON с шестью селекторами: `name`, `price`, `currency`, `pictures`, `category`, `article`.

Каждый сгенерированный селектор тут же проверяется на исходной странице через `BeautifulSoup`. Если ничего не находит — `null` и пометка в `markdownComments`. Это защита от галлюцинаций модели, которая иногда придумывает классы вроде `.product-deluxe-superpremium`, которых на странице нет.

Валидированная разметка ложится в SQLite через UPSERT по ключу домена. Дальше любые URL того же магазина обрабатываются модулем `applier` без вызова LLM — обычный `BeautifulSoup` плюс нормализация цены и артикула. Это даёт скорость классических парсеров и универсальность AI: одна генерация покрывает тысячу страниц.

Экспорт в TOON через GET `/api/selectors/export.toon` формирует файл по эталонной структуре из задания `Parsmart`. Сериализатор поддерживает все нужные конструкции: нумерованные массивы (`domains[N]`, `markdownComments[N]`), вложенные объекты (`productPage`, `cartPage`, `orderPage`), стабильный порядок полей, экранирование кавычек.

5.4. Интерактивные действия

Эту фичу попросил руководитель на финале. По задаче: сервис должен не только читать страницу, но и нажимать кнопки, добавлять в корзину. Универсально, без жёстких правил под конкретный магазин.

Сначала Даниил написал `element_finder` с системой эвристического скоринга. Каждый потенциально-кликабельный элемент (`button`, `a`, `input[type=submit]`, `div` с `role=button` или `onclick`) получает баллы по нескольким признакам. Точное совпадение текста — 20 баллов, частичное — 15, все слова встречаются — 8. Класс или `id` под регулярку — 25. `role=button` — 5. Тег `button` — 10, `input[type=submit]` — 8, `a` — 4. `Anti-patterns` («избранное», «wishlist», «compare») — минус 50. `Disabled`-кнопки — минус 30.

Многоязычность через словарь фраз для каждого намерения. Для `add_to_cart` поддерживаются русские формулировки («в корзину», «купить», «заказать»), английские («add to cart», «buy now», «purchase»), французские и немецкие. Новый язык — одна строка в словаре.

Минимальный порог уверенности — 30 баллов. Меньше — Михаил в `interactor` дёргает LLM-fallback: модель получает очищенный HTML и возвращает JSON с предложенным селектором, `confidence` от 0 до 100 и обоснованием. Если `confidence` ниже 50 — возвращаем `not found` без попыток клика наугад. Это важно: лучше честно сказать «не нашёл», чем кликнуть не туда.

Сборка надёжного селектора (функция `build_selector`) идёт по приоритетам: `id` если не похож на хэш, потом `data-testid`, `data-action`, `input[name=...]`, потом `tag` с осмысленными классами, в крайнем случае путь от ближайшего предка с `id`. Игнорируем хэш-классы CSS-in-JS (`css-1a2b3c`), `styled-components` (`sc-aBcD1`), `Next.js` (`jsx-1234567890`), `Material-UI` (`MuiButton-root-12345`).

Сам клик выполняется через `Playwright`. Все шаги пишутся в лог: `goto`, `page_loaded`, `heuristic`, `candidate` (с `score`), `llm` (если был fallback), `click`, `post_click_wait`, `done`. Каждый шаг с `timestamp_ms` — потом можно посмотреть в БД историю взаимодействия и понять, что произошло.

6. Тестирование

6.1. Методология

Подход трёхуровневый. `Unit` — отдельные функции без сети и БД. `Integration` — реальный FastAPI и временная SQLite, но `Playwright` и LLM API замочены через `unittest.mock`. `End-to-end` — ручные прогоны через работающий сервер на синтетических фикстурах и реальных сайтах.

Всего 139 автотестов в 10 модулях. Полный прогон — секунда, потому что внешние API замочены. Это позволяет крутить тесты в CI на каждом коммите без подключения к интернету.

6.2. Что покрыто

`test_schemas.py` (16 тестов): Pydantic-валидация. URL должен начинаться с `http(s)`, не быть пустым, не длиннее 2048 символов. Цена положительная и реалистичная (меньше 10 млн). Пустое название становится `None`.

`test_cleaner.py` (11): `script` и `style` удаляются, у `img` сохраняется `src`, пустые контейнеры выкидываются, длинный HTML усекается.

test_llm_parser.py (12): _extract_json парсит чистый JSON, JSON в markdown, с trailing comma, с null. Нормализация «1 990₽» → 1990.0 и альтернативных имён полей (name вместо title).

test_applier.py (19): валидация селекторов отбрасывает несуществующие. Apply извлекает поля и абсолютизирует URL. Цена парсится в нескольких форматах. Артикул нормализуется корректно.

test_toon.py (12): пустой список → domains[0], null-селекторы не попадают в выход, комментарии нумеруются с 1, кавычки экранируются, порядок полей productPage стабильный.

test_database.py (8): save и get parsed_products, обработка error без полей, корректная история, raw_response сохраняется.

test_api.py (13): /health, /api/results, /api/parse с моками, ошибка fetch возвращается со статусом success+error, 404 на несуществующем id, /api/interactions, очистка.

test_selectors_api.py (9): невалидный URL → 422, успех с моками, отбрасывание галлюцинированных селекторов, apply без LLM, list, export.toon.

test_element_finder.py (20): русские/английские/французские/немецкие кнопки, anti-patterns, build_selector с priopritetami, _looks_autogenerated для всех видов хэш-классов.

test_interact_api.py (9): неправильный URL и action → 422, success/not_found/error, шорткат /cart/add, история runs.

6.3. Локальная фикстура

Файл *tests/fixtures/sample_product.html* повторяет структуру эталонной страницы из задания Parsmart — со всеми классами из примера. Прямое извлечение через LLM возвращает *title* «Электрогриль Tefal OptiGrill Elite GC750D30» (точное совпадение с эталоном), *article* «2006 4875», корректный *image_url*. Генерация селекторов возвращает *name* «.m-product .main .name» и *pictures* «#m-zoomable-image picture img» — оба ТОЧНО совпадают с эталоном из задания. Применение в режиме *apply* нормализует артикул: «2006 4875» → «20064875».

Вторая фикстура *product_with_buttons.html* содержит четыре разных кнопки: «В корзину», «В избранное» (*anti-pattern*), «Купить сейчас», «Закончился» (*disabled*). На *action=add_to_cart* эвристика находит *#addToCartBtn* со *score* 55.0. LLM не вызывается (выше порога). Клик через Playwright реально срабатывает — JS-обработчик меняет *document.title* с «Кофемашина Bosch TKA8633 — Sample Shop» на «Кофемашина Bosch TKA8633 — добавлено в корзину». Это подтверждает, что наша система не просто отправляет *click event*, а реально взаимодействует со страницей.

6.4. Реальные сайты

6.4.1. books.toscrape.com

Официальный демо-сайт от создателей Scrapy. На странице книги «A Light in the Attic» все три режима работают. Прямое извлечение возвращает корректный *title* и UPC-код книги (a897fe39b1053632). Генерация селекторов даёт все шесть полей без единого предупреждения: *name* «.product_page h1», *price* «.product_main .price_color», *article* из таблицы характеристик. Главное — применение этих же селекторов к ДРУГОЙ книге того же домена («Tipping the Velvet») без повторного вызова LLM работает идеально: *title* «Tipping the Velvet», *price* 53.74, *article* «90fa61229261140a». Это и есть Parsmart в действии — один вызов модели покрывает тысячи страниц.

6.4.2. Wildberries

Крупнейший российский маркетплейс с антибот-защитой. Stealth-патчи позволили получить нормальный HTML страницы (1.6 МБ — с настоящим контентом, не SPA-shell). Интерактивное действие *add_to_cart* на */catalog/192222089/detail.aspx* сработало. Эвристика дала *score* 29 — ниже порога 30, автоматически переключились на LLM-fallback. GPT-4o-mini вернул селектор *.product-card .add-basket.j-add-to-basket* с *confidence* 90 и обоснованием «This element has a clear class name indicating it is the add-to-basket button». Клик прошёл за 16 секунд.

6.4.3. demoblaze.com

Тестовый магазин, специально созданный для автоматизации. Эвристика обнаружила кандидата со *score* 24 (ниже порога), сработал LLM-fallback. GPT-4o-mini нашёл *.product-content .btn.btn-success* с *confidence* 90. Клик прошёл.

6.4.4. Где не получилось

На Ozon stealth не помог — возвращается «Antibot Captcha» от DataDome. На DNS-shop вообще HTTP 403 от Akamai на основе TLS-fingerprint. Это известные ограничения headless-парсинга без резидентных прокси-сетей, общие для всех self-hosted решений. В обзоре аналогов мы упоминаем, что даже Diffbot и Apify арендуют именно резидентные прокси для обхода такой защиты. Для курсового проекта без бюджета на прокси это вне досягаемости — и мы это честно зафиксировали.

На М.Видео stealth прошёл, страница загрузилась, но контент карточки догружается JavaScript-ом из внутреннего API через несколько секунд после networkidle. Наш HTML-snapshot

снимается до полной отрисовки. Параметр `extra_wait_ms` на 5–10 секунд это решает, но сильно замедляет общий прогон.

На `books.toscrape.com` странице книги система корректно вернула `not_found` на запрос `add_to_cart` — кнопки добавления в корзину там действительно нет (она только в каталоге). И эвристика, и LLM сошлись на том, что элемента нет. Это важный показатель — система не галлюцинирует, а честно признаёт неудачу.

7. Заключение

Что получилось. Универсальный парсер интернет-магазинов работает в трёх режимах. Подбирает CSS-селекторы через LLM, сохраняет их в TOON-формате как в задании Parsmart, применяет потом к тысячам страниц без обращения к модели. Умеет нажимать кнопки и добавлять товары в корзину автоматически — это требование руководителя мы закрыли на финале.

Что покрыто. REST API на FastAPI с автогенерируемым Swagger UI. Загрузчик на Playwright со stealth-патчами, проходит антибот Wildberries и Citilink. Препроцессор HTML с двумя режимами очистки. LLM-парсер с поддержкой Llama 3.1 и GPT-4o-mini, с fallback на прямой Hugging Face API при проблемах в LangChain. Универсальный поиск кнопок через скрининг с поддержкой четырёх языков плюс LLM-резерв. Pydantic-валидация всех контрактов. Асинхронное хранилище на aiosqlite с тремя таблицами. 139 автотестов, прогон за секунду. Docker-сборка. Открытый GitHub-репозиторий.

Главное архитектурное решение — кэшировать селекторы, а не сами данные. Один вызов LLM на домен покрывает тысячи страниц. На `books.toscrape.com` мы наглядно это показали: сгенерировали селекторы по странице «A Light in the Attic», применили к «Tipping the Velvet» — обе книги корректно распарсились, LLM второй раз не вызывался.

Что не получилось. Не прошли антибот Ozon и DNS-shop — там DataDome и Akamai с TLS-fingerprint. Без резидентных прокси-сетей такую защиту не обойти, и это вне рамок self-hosted решения. Также на сложных страницах маркетплейсов (Wildberries iPhone — 30+ продавцов одного товара в DOM) LLM иногда выбирает не главную карточку, а блок альтернативных предложений. Это решается специализированным промптом под конкретный сайт, но мы решили не отступать от принципа универсальности.

Куда расти. Расширить тестовый датасет до 200–300 страниц с ground-truth разметкой. Собрать benchmark на нескольких моделях (Llama, Mistral, Qwen, GPT, Claude). Добавить генерацию селекторов для cartPage и orderPage — сейчас только productPage. Сделать систему автоматической детекции редизайнов с триггерной регенерацией селекторов. Перейти на PostgreSQL и добавить Redis-кэш для прод-нагрузок. Поддержат multistep flows в интерактивных действиях — выбрать размер, потом цвет, потом добавить в корзину.

8. Список литературы

Brown T. et al. Language Models are Few-Shot Learners // *Advances in Neural Information Processing Systems*. — 2020. — Vol. 33. — pp. 1877–1901.

Liu P. et al. Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing // *ACM Computing Surveys*. — 2023. — Vol. 55, No. 9. — pp. 1–35.

Touvron H. et al. Llama 2: Open Foundation and Fine-Tuned Chat Models // *arXiv:2307.09288*. — 2023.

OpenAI. GPT-4 Technical Report // *arXiv:2303.08774*. — 2023.

Ramírez S. FastAPI Documentation. — Tiangolo, 2024. — URL: <https://fastapi.tiangolo.com/>

Microsoft Corporation. Playwright Documentation. — 2024. — URL: <https://playwright.dev/python/>

Beautiful Soup Documentation. Crummy Software. — 2024. — URL: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

Pydantic v2 Documentation. — Pydantic Services Inc, 2024. — URL: <https://docs.pydantic.dev/>

LangChain Documentation. — LangChain Inc, 2024. — URL: <https://python.langchain.com/>

Hugging Face Inference API Documentation. — Hugging Face Inc, 2024. — URL: <https://huggingface.co/docs/api-inference/>

Mitchell T.M., Cohen W.W. Never-Ending Learning // *Communications of the ACM*. — 2018. — Vol. 61, No. 5. — pp. 103–115.

SQLAlchemy and aiosqlite Documentation. — 2024. — URL: <https://docs.sqlalchemy.org/>, <https://aiosqlite.omnilib.dev/>

Diffbot AI-powered Web Data Extraction. — 2024. — URL: <https://www.diffbot.com/>

ScrapingBee Web Scraping API. — 2024. — URL: <https://www.scrapingbee.com/>

Apify Web Scraping Platform Documentation. — 2024. — URL: <https://docs.apify.com/>

Octoparse Visual Web Scraping. — 2024. — URL: <https://www.octoparse.com/>

Scrapy 2.11 Documentation. — Scrapinghub Ltd, 2024. — URL: <https://docs.scrapy.org/>

Playwright Stealth Plugin (Python). — Matwmaster58, 2024. — URL: https://github.com/Matwmaster58/playwright_stealth

Постановка задачи Parsmart 2025. — Курсовой проект студентов НИУ ВШЭ, 2025.

ITESCIA Tech. DataDome and Antibot Defense Mechanisms. — Technical Report, 2023.